UCS503P – Software Engineering

TIET LostLink

A Secure Campus Lost-and-Found System

Submitted by:

1024031152	Kiesha Kapoor
1024030804	Aarush Prabhakar
1024030796	Kasvi Bhatia
1024030810	Gurleen Kaur

Group: 3c55 BE Third Year, CSE

Submitted to:

Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

August 18, 2026

Contents

1 Introduction **2**
1.1 Background and Context 2
1.1.1 Problem Statement 2
1.2 Project Objectives 2

2 Methodology and Design **4**
2.1 System Architecture 4
2.1.1 High-Level Design 4
2.1.2 Technical Stack 4
2.2 Implementation Details 5
2.2.1 Module 1: Core Functionality 5

Chapter 1

Introduction

1.1 Background and Context

Students at the Thapar Institute of Engineering and Technology (TIET) frequently lose personal belongings across the campus. These include bags, water bottles, wallets, keys, books, and electronic accessories. Information about lost and found items is currently shared through fragmented channels: class WhatsApp groups, hostel groups, social-media stories, and informal word of mouth. The campus is large, with several academic blocks, hostels, activity areas, and food outlets. As a result, a lost-item report and a corresponding found-item report often never reach the same students.

This project addresses that gap. It centralises lost-and-found activity into a single, searchable, mobile-friendly web platform. The broader goal is to improve resource recovery, trust, and student convenience. This reduces the number of usable personal items that remain unclaimed or are unnecessarily replaced. The immediate engineering objective is a simple, secure, and deployable web application. It centralises reports, suggests possible matches, supports ownership claims, and records the successful return of items. The system assists students in identifying possible matches. It does not automatically declare a person to be the legal owner of an item.

1.1.1 Problem Statement

The prevailing informal process creates several concrete problems:

- **Fragmented information:** Lost- and found-item reports posted in different groups may never reach the same audience.
- **Poor searchability:** Existing posts cannot be filtered by item category, date, colour, brand, or campus location.
- **Low persistence:** Lost-and-found messages quickly disappear below newer conversations.
- **Repeated posts:** The same information must be shared across multiple groups.
- **Weak ownership verification:** Publicly displaying every identifying detail allows an unrelated student to make a false claim.
- **No structured handover process:** There is no consistent method for submitting claims, checking private information, approving a claim, and confirming that an item was returned.

A centralised platform addresses each of these problems. It keeps sensitive ownership details private while still enabling matching.

1.2 Project Objectives

1. **Centralised reporting and search:** Provide a single platform for authenticated students to submit structured lost- and found-item reports. Reports can be filtered by category, date, status, colour, brand, and location. Search results should return within approximately two seconds during the pilot.

2. **Explainable match suggestions:** Implement a lightweight, weighted, and explainable matching algorithm that ranks candidate reports. The target is a top-five match-suggestion success rate of at least 75% on a controlled test set.
3. **Secure ownership verification and handover:** Support private ownership claims, finder approval, and a one-time handover code. Unauthorised users must not be able to view private item details or claim evidence.

These objectives are framed to be Specific, Measurable, Achievable, Relevant, and Time-bound (SMART). They are designed to be delivered by a two-person team across four weekly iterations. Each iteration produces a demonstrable improvement.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

TIET LostLink follows a conventional three-tier web architecture. It cleanly separates presentation, application logic, and data. This structure keeps the implementation practical within the course timeline. It also avoids the need for a separate machine-learning service.

- **Presentation tier:** A React (Vite) single-page application provides the student and administrator interfaces. These include the reporting forms, report feed, search and filter controls, claim submission, and handover screens.
- **Application tier:** A Node.js and Express.js backend exposes stateless REST APIs and enforces the business rules. These cover authentication checks, the weighted matching algorithm, claim authorisation, one-time handover-code generation, and administrator moderation.
- **Data tier:** Supabase provides managed PostgreSQL for relational data, Supabase Authentication for institutional-email login, and Supabase Storage for item and claim photographs.

The simplified request path is:

$$\textit{React Frontend} \rightarrow \textit{Express Backend} \rightarrow \textit{Supabase PostgreSQL}$$

Several decisions keep future scaling cheap. The frontend and backend stay separate. The REST APIs are stateless. Images are stored independently of report data. Growth is then handled through pagination and database indexing, without architectural change.

2.1.2 Technical Stack

- **Framework/Language:** React with Vite (frontend); Node.js with Express.js (backend); JavaScript throughout.
- **Database:** Supabase PostgreSQL, with Supabase Authentication and Supabase Storage for managed auth and image hosting.
- **Tools:** Git and GitHub for version control and pull-request review; a basic GitHub Actions workflow for continuous integration; Postman for manual API testing; Vercel and Render for deployment.
- **Libraries:** React Router for navigation; Axios (or the Fetch API) for client–server communication; basic CSS or Tailwind CSS for styling; Jest and Supertest for backend testing.

This stack was selected for practical reasons. It is widely used and well documented. It reduces authentication and image-storage development time. It requires no separate machine-learning server. It deploys on free or low-cost services. It is realistic for a two-person team working within four weeks.
The relational schema underpinning the data tier is organised into seven core tables: *Users*, *ItemReports*, *Matches*, *Claims*, *Handovers*, *Notifications*, and *Flags*. These separate identity,

Component	Technology
Frontend	React with Vite
Styling	Basic CSS or Tailwind CSS
Routing	React Router
Backend	Node.js and Express.js
Database	Supabase PostgreSQL
Authentication	Supabase Authentication
Image storage	Supabase Storage
API communication	Axios or Fetch API
Backend testing	Jest and Supertest
API testing	Postman
Frontend deployment	Vercel
Backend deployment	Render
Version control	Git and GitHub
Continuous integration	Basic GitHub Actions workflow

Table 2.1: Technical stack for TIET LostLink

report content, suggested pairings, ownership claims, handover records, in-application notifications, and moderation flags respectively.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The core module implements the primary reporting-and-matching workflow. This workflow is the foundation of the platform and the first deliverable of the project. It covers authentication, structured lost- and found-item reporting with image upload, the searchable report feed, and the weighted possible-match suggestion engine.

Authentication and reporting. Students register and log in using an institutional email address through Supabase Authentication. Once authenticated, a student submits either a lost-item or a found-item report through a structured form. The form captures item category, description, date, approximate location, colour, brand, and an optional photograph. A finder submits similar information while keeping certain identifying characteristics private. On submission, the backend persists the record to the *ItemReports* table and generates a unique report ID. Uploaded images are stored separately in Supabase Storage, with the report holding only an image reference. This keeps report data lightweight and supports future scalability.

Search and report feed. The report feed presents active reports. It can be filtered by category, date, status, colour, brand, and location. This directly replaces the poor searchability of informal channels. Reports are stored relationally, with indexes on category, report type, date, status, and location. Filtering therefore remains responsive, and results are paginated to bound query cost.

Possible-match algorithm. When a report is submitted, the system compares it against active reports of the *opposite* type within the *same* item category. Candidates first pass a set of mandatory filters. The report types must be opposite (one Lost, one Found). Both must belong to the same category. Both must have an active status. Surviving candidates receive a weighted

score, expressed as

$$M = C + B + R + L + D + T,$$

where C is category matching, B is brand matching, R is colour matching, L is location matching, D is date matching, and T is description keyword similarity. The individual weights are summarised in Table 2.2.

Matching condition	**Score**
Same item category	30 points
Same brand	15 points
Same or similar colour	15 points
Same or nearby campus location	15 points
Dates within three days	15 points
Description keyword similarity	10 points
Maximum possible score	**100 points**

Table 2.2: Weighted scoring used by the possible-match algorithm

The resulting score is interpreted in three bands. A score of $M \geq 75$ is a high-likelihood candidate. A score of $50 \leq M < 75$ is a possible candidate. A score below 50 is a weak candidate. These labels only assist users in reviewing reports. They never prove ownership.

The description-similarity component T uses lightweight text processing. Each description is converted to lowercase, stripped of punctuation, and split into keywords. Common stopwords such as “the”, “a”, “and”, and “near” are removed. The score then reflects the percentage of meaningful keywords shared between the two descriptions. Consider a lost report reading “Black Sony earbuds with a scratched case” and a found report reading “Sony wireless earbuds in a black charging case”. The overlap is *sony*, *earbuds*, *black*, *case*. That is enough signal to rank the pair, without a language model or a separate Python service. The ranked list is then returned to the student for review. The ownership-claim and handover workflow that follows is documented in Module 2.

2.3 Design Diagrams

This section collects the data-flow, UML, and workflow diagrams that describe TIET LostLink. Figure 2.1 shows the context-level data flow diagram (DFD Level 0), and Figure 2.2 decomposes it into the six main processes (DFD Level 1). Figure 2.3 shows the report-submission and match-finding interaction as a sequence diagram, and Figure 2.4 shows the same interaction as a collaboration diagram. Figure 2.5 shows the component structure of the system. Figure 2.6 shows the end-to-end workflow as a single flow, and Figure 2.7 shows the same workflow across the student, system, finder, and administrator roles. Figure 2.8 shows the class structure of the system, and Figure 2.9 shows the lifecycle states of an item report.

2.3.1 Context Level Diagram (DFD Level 0)

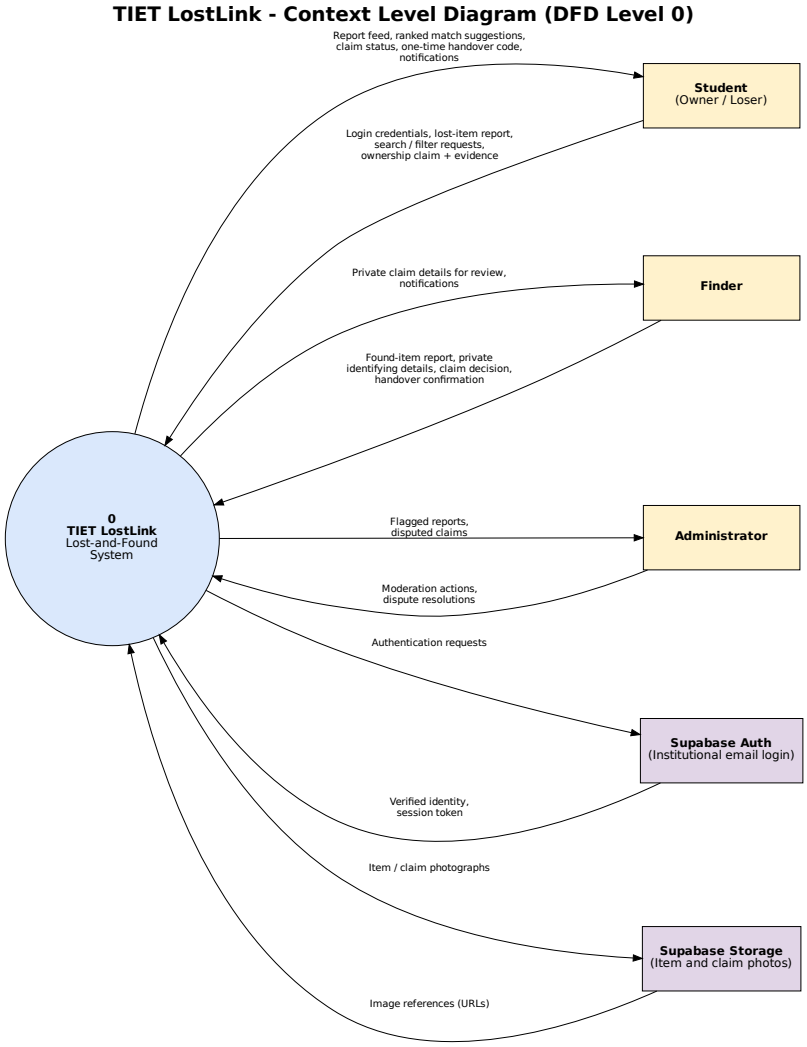


Figure 2.1: Context level data flow diagram of TIET LostLink.

2.3.2 Data Flow Diagram (Level 1)

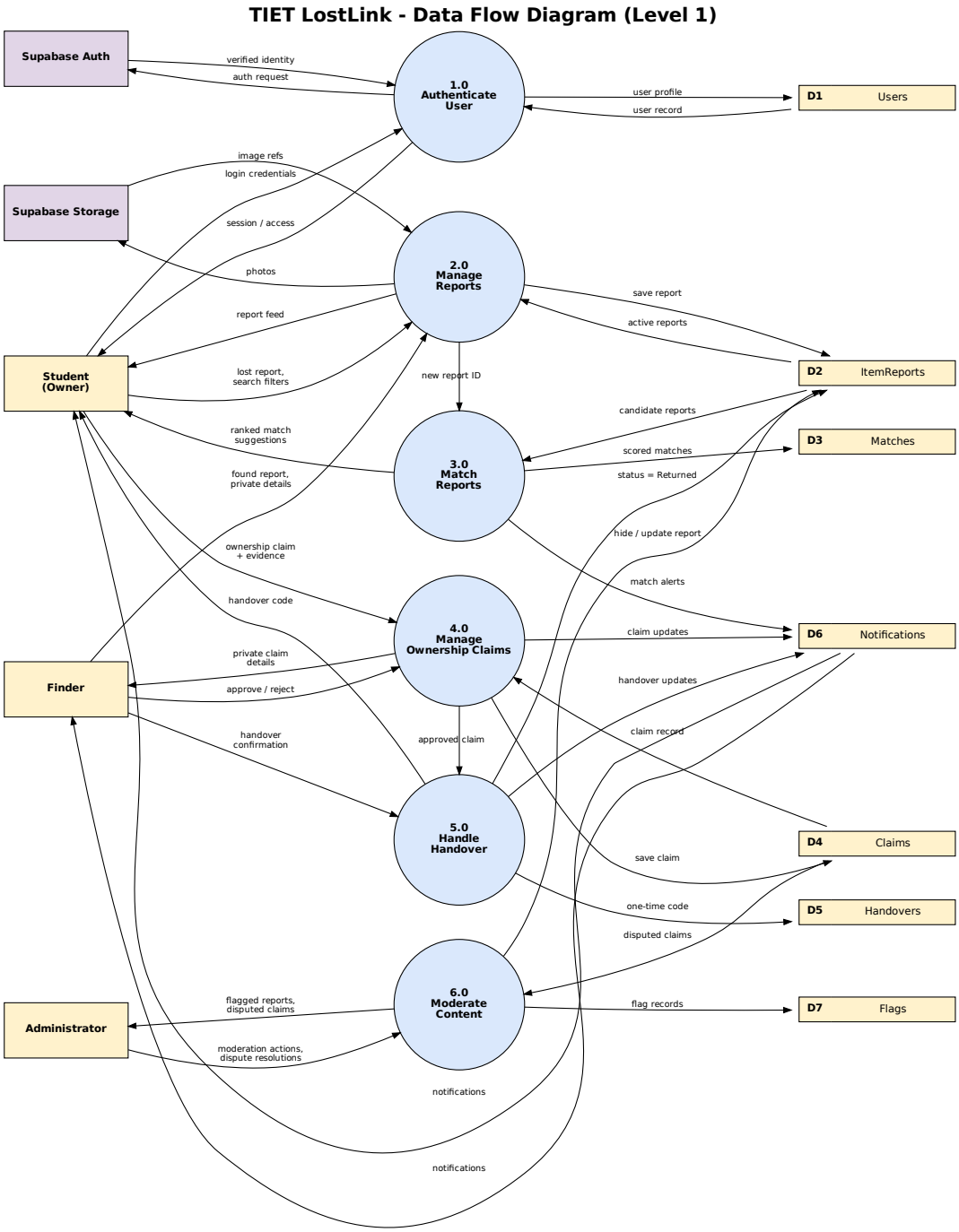


Figure 2.2: Level 1 data flow diagram of TIET LostLink showing the six main processes and seven data stores.

2.3.3 Sequence Diagram: Submit Report and Find Matches

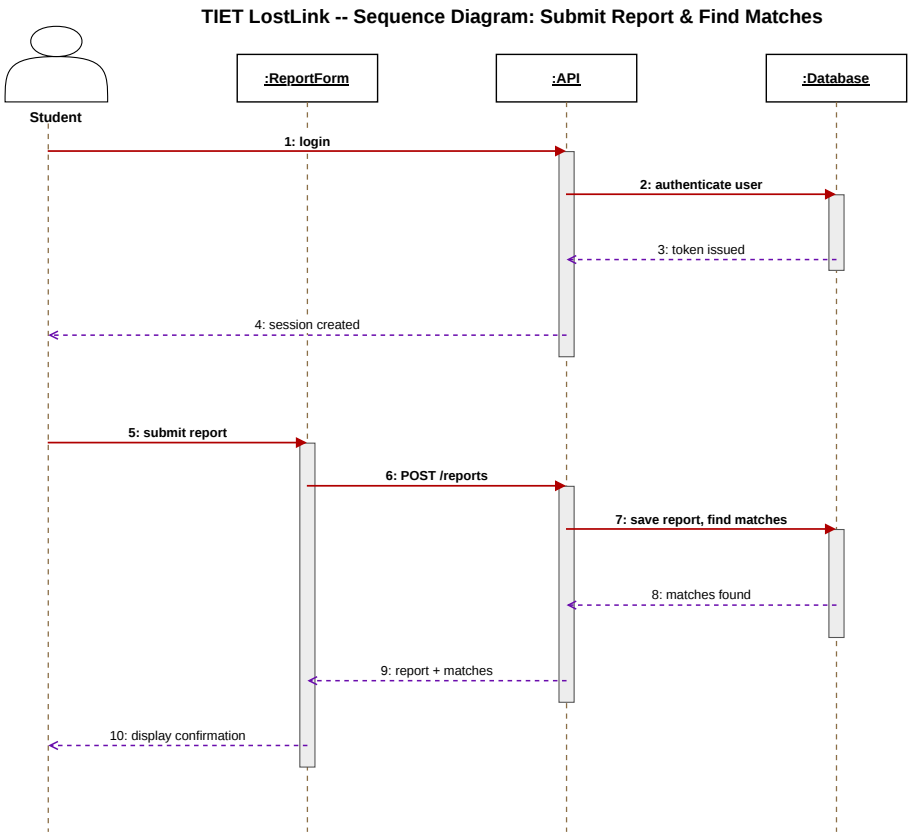


Figure 2.3: Sequence diagram for submitting a report and finding matches.

2.3.4 Collaboration Diagram: Submit Report and Find Matches

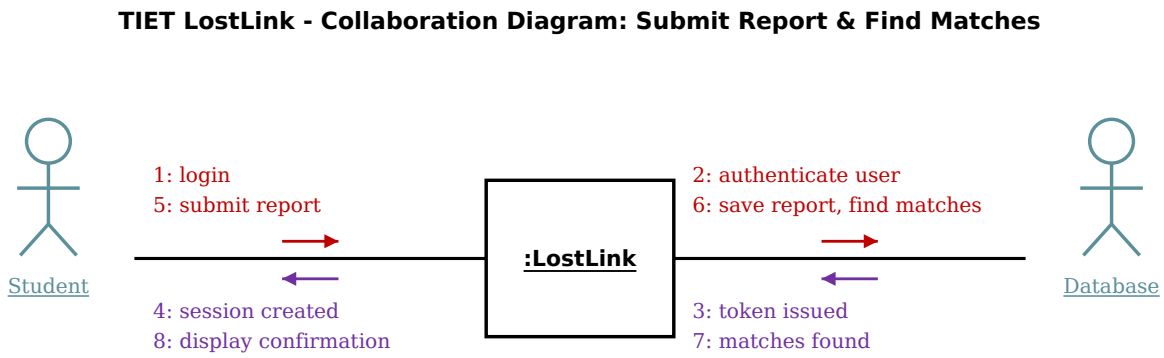


Figure 2.4: Collaboration diagram for submitting a report and finding matches. Red arrows show requests and purple arrows show responses.

2.3.5 Component Diagram

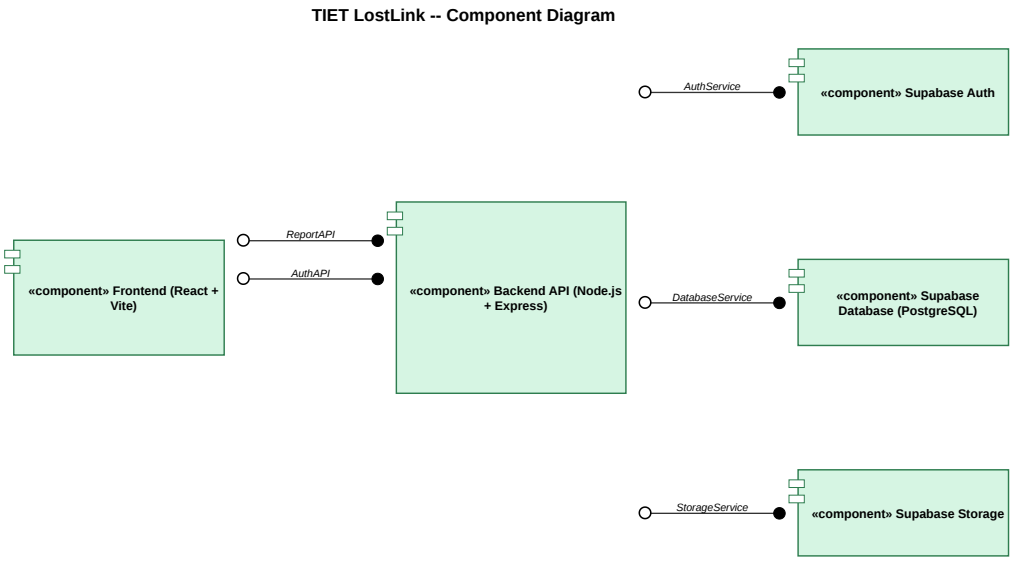


Figure 2.5: Component diagram of TIET LostLink showing the frontend, backend API, and Supabase services.

2.3.6 Activity Diagram (Single Flow)

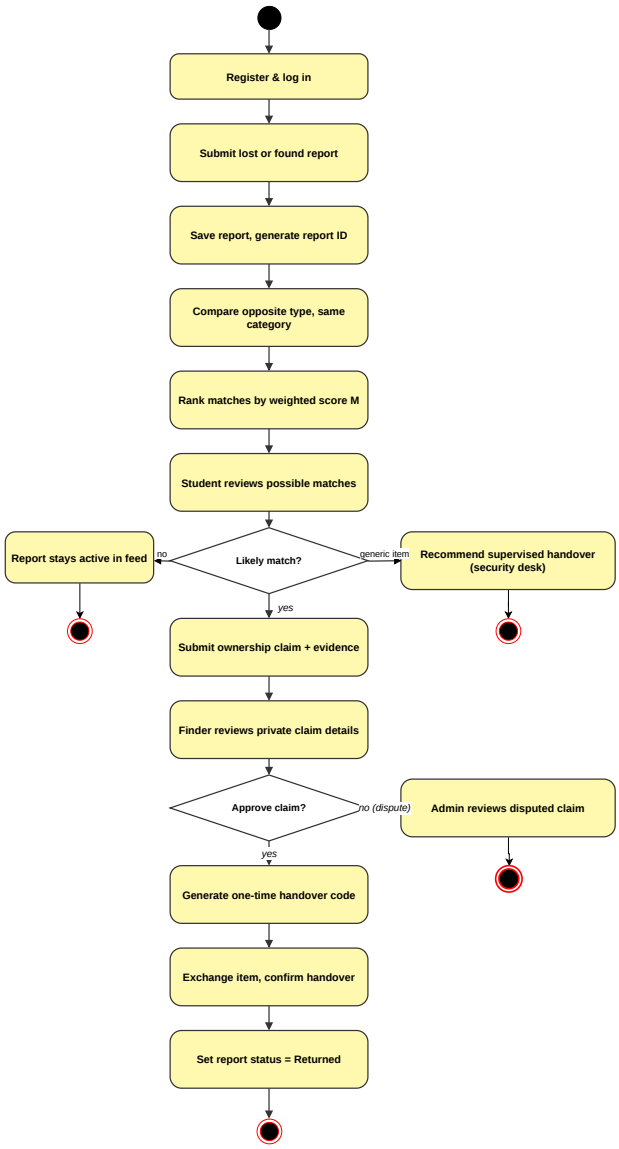


Figure 2.6: Activity diagram of the LostLink workflow without swimlanes.

2.3.7 Activity Diagram with Swimlanes

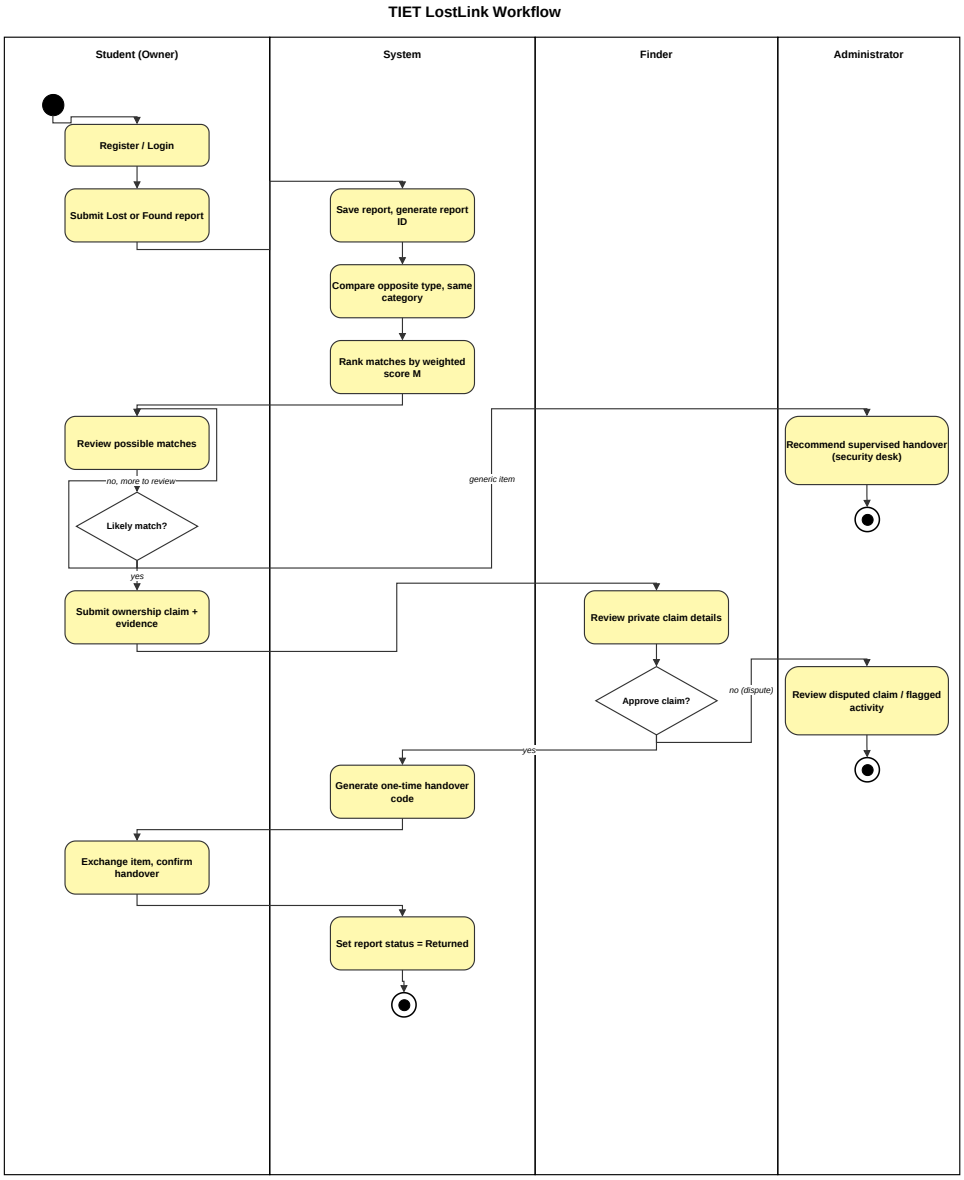


Figure 2.7: Activity (swimlane) diagram of the LostLink workflow from report submission to confirmed handover.

2.3.8 Class Diagram

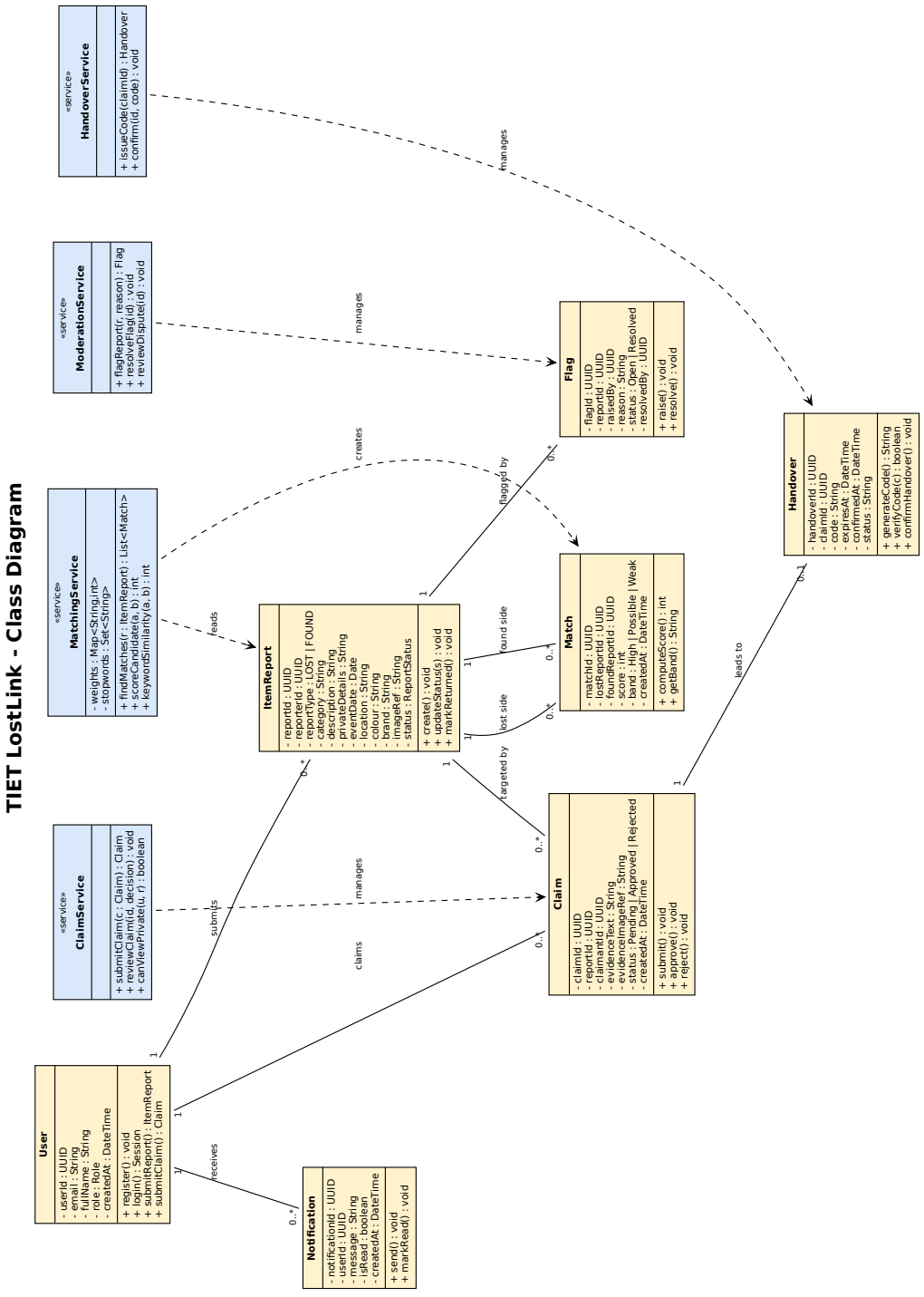


Figure 2.8: Class diagram of TIET LostLink showing the entity classes, service classes, and their relationships (rotated for legibility).

2.3.9 State Diagram: Item Report Lifecycle

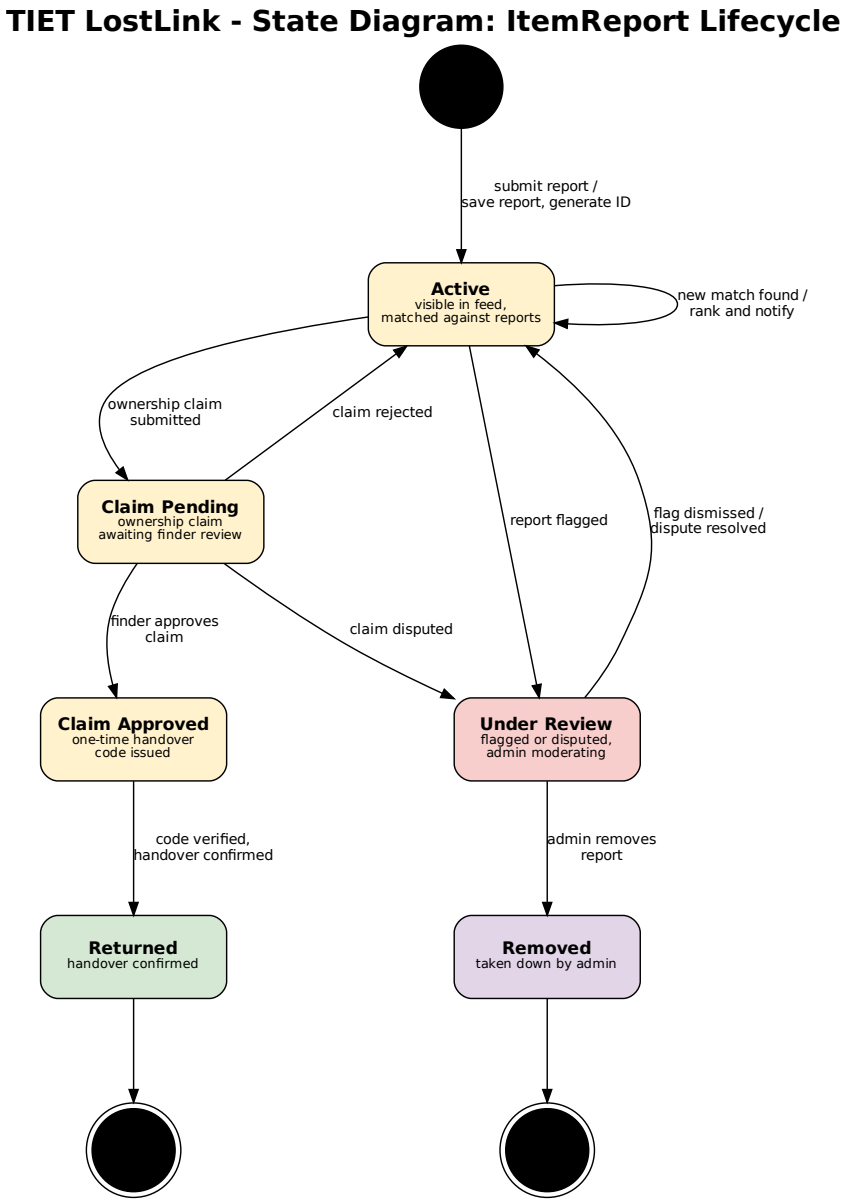


Figure 2.9: State diagram of an item report, from submission to return or removal.

2.3.10 Use Case Diagram

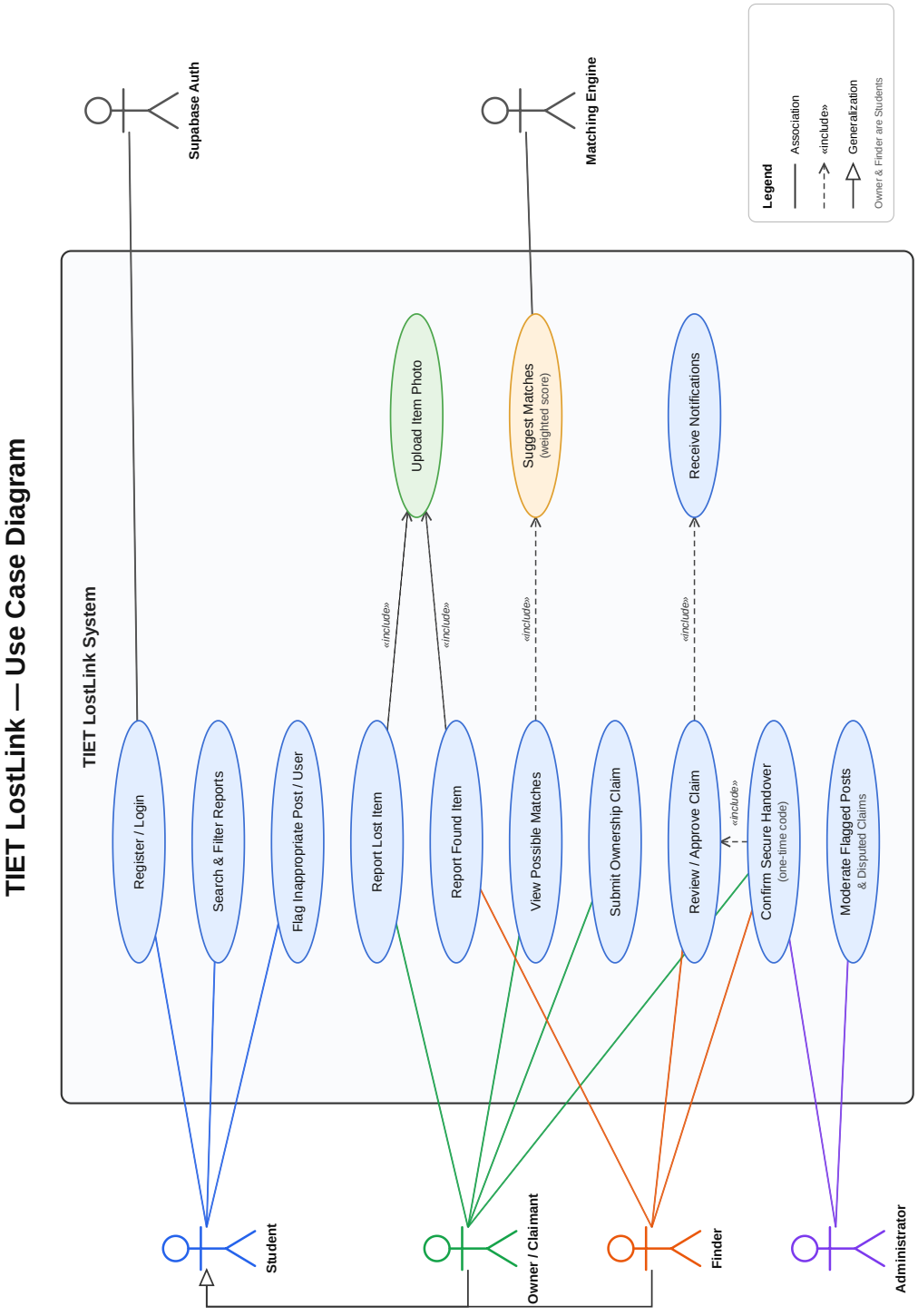


Figure 2.10: Use case diagram of TIET LostLink (rotated for legibility).